CS7CS4/CSU44061 Machine Learning
Student Name: Stephen Komolafe
Student Number: 21336975

Dataset ID: # id:8--16—8

Week 2 Assignment - Logistic Regression & SVM

Part A: Logistic Regression

(i) Figure 1 provides a 2D visualisation of the dataset in the form of a scatter plot, where each point corresponds to one row of input feature values (the first two columns of the data). The x-axis represents the first feature value, and the y-axis represents the second feature value. Data points are labelled according to their ±1 valued target value (the third column of the data): points from the positive class (+1) are shown in blue, while points from the negative class (−1) are shown in red, with a black outline given to make each point clearly distinguishable. A legend is anchored to the upper-right corner of the window so as not to obscure the points on the plot.

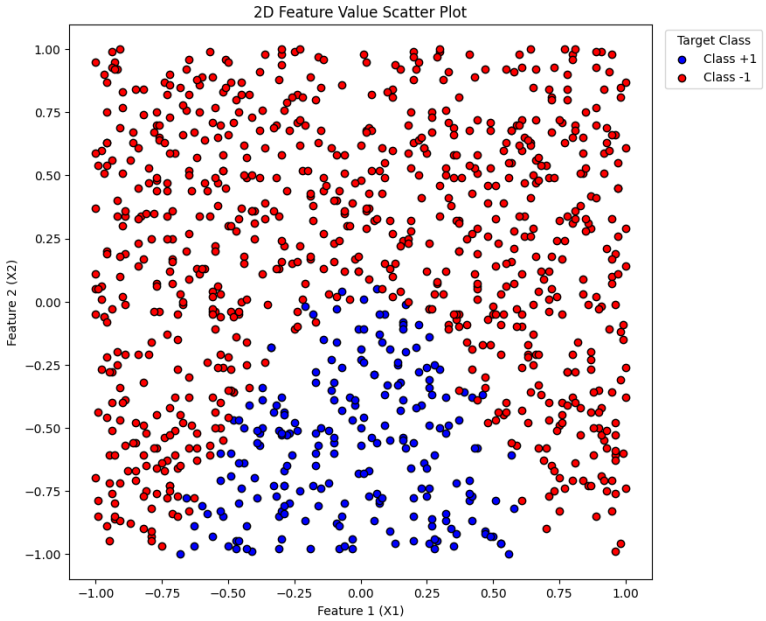

Figure 1. Visualisation of Dataset

From this plot, it can be seen how the two classes are distributed in relation to each other. The data points appear well separated, with the distinct areas where class +1 and class -1 markers overlap suggesting that a curved decision boundary may be able to distinguish these classes more effectively.

(ii) A logistic regression classifier from the sklearn.linear_model library was trained on the data:

```
from sklearn.linear_model import
LogisticRegression
Xtrain = file.iloc[:,0:2] #col1&2
model = LogisticRegression().fit(Xtrain, y)
```

Where `Xtrain` is a 999 x 2 vector containing both feature 1 and 2 input values read from the dataset, making each row of `Xtrain` one data point on the plot in figure 1.

The logistic regression model is expressed as a sigmoid function applied to a linear combination of the two features. The logistic regression classifier has the following prediction model:

$$\hat{y} = \sigma(\theta^T x)$$

Where:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

θ_0: Intercept (Bias), θ_1: Feature 1 Coefficient, θ_2: Feature 2 Coefficient

Which is expressed as a sigmoid function:

$$\sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2)}}$$

The parameter values obtained from training the model are $\theta_0 = -2.14$, $\theta_1 = 0.123$, and $\theta_2 = -3.48$. From this the following can be inferred:

$$\theta^T x = -2.14 + 0.123 x_1 - 3.48\, x_2$$

$$\hat{y} = \sigma(\theta^T x) = \frac{1}{1 + e^{-(-2.14 + 0.123 x_1 - 3.48\, x_2)}}$$

Feature 1 has a small positive coefficient, so higher values of Feature 1 would increase the probability of predicting the positive class (+1). However, the large negative coefficient for Feature 2 indicates that as Feature 2 increases, the probability of predicting class +1 decreases significantly. This makes Feature 2 the dominant factor influencing the model's predictions, while Feature 1 plays only a minor role, as it is closer to zero. The negative intercept shifts the decision boundary. Since it is negative, the model has an overall tendency to predict class -1 unless the weighted sum of features is large enough to offset the intercept.

(iii) The decision boundary of the logistic regression model is the set of points where the model is equally likely to predict either class, that being:

$$P(y = 1|x) = P(y = 0|x) = \frac{1}{2}$$

Given that:

$$P(y = 1|x) = \frac{1}{1 + e^{-\theta^T x}} \qquad (Where\ \theta = \theta_0, \theta_1, \theta_2)$$

$$\therefore \frac{1}{1 + e^{-\theta^T x}} = \frac{1}{2}$$

$$\therefore 1 + e^{-\theta^T x} = 2$$

$$\therefore e^{-\theta^T x} = 1$$

$$\therefore \theta^T x = 0$$

From this the linear equation can be solved to get the decision boundary:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$$

$$\theta_2 x_2 = -\theta_0 - \theta_1 x_1$$

$$\Rightarrow x_2 = -\frac{\theta_0 + \theta_1 x_1}{\theta_2}$$

Using this equation, the decision boundary could now be implemented using the following code:

```
x1_vals = np.linspace(Xtrain.iloc[:,0].min(), Xtrain.iloc[:,0].max())
x2_vals = -(theta0 + theta1 * x1_vals) / theta2
```

Here `x1_vals` is a smooth range of 100 evenly spaced feature 1 (x1) values that span the full extent of the dataset from the smallest to the largest training value. These are then substituted into the decision boundary equation to calculate the matching feature 2 (x2) values, giving two arrays (x1, x2) that can be plotted as a dashed line and added to the scatter plot.

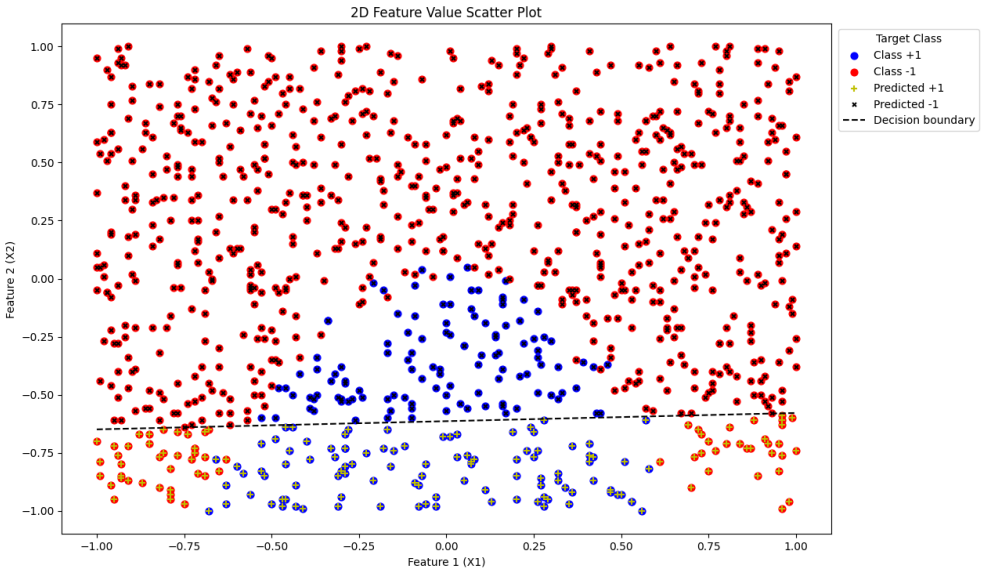

Figure 2. Logistic Regression Model - Predicted Classes & Decision Boundary

Figure 2 shows the original training data (red and blue points), the model's predictions (black and yellow markers), and the logistic regression decision boundary (dashed line) that best separates the two classes based on the learnt parameters.

(iv) The predictions largely align with the training data, showing that the logistic regression model has captured the main separation between the two classes. The linear decision boundary is appropriate here because most of the positive and negative examples are separated along Feature 2, and a straight line provides a reasonable split. However, as mentioned in a(i) a curve is visible where the classes overlap into each other, meaning a linear decision boundary cannot perfectly separate them. This would explain the misclassified points at the left and right ends below the boundary and misclassified points above the boundary.

Part B: SVM

(i) The SVM model for predictions is as follows:

$$y_{SVM} = h_\theta(x) = sign(\theta^T x)$$

$$\therefore y_{SVM} = sign(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Meaning depending on the value of $(\theta^T x)$ the outputs can be -1, 0, or +1.

Linear SVM classifiers were trained for penalty parameters C ranging from 0.001 up to 1000 in $x10$ increments using the following code:

```
from sklearn.svm import LinearSVC
C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
for i, C in enumerate(C_values):
    svm_model = LinearSVC(C=C, max_iter=10000).fit(Xtrain, y)
    print(f"\nC = {C}")
    print("Bias (θ0):", svm_model.intercept_)
    print("Weights (θ1, θ2):", svm_model.coef_)
```

Here, `LinearSVC` constructs a linear support vector classifier with penalty parameter C. The argument `max_iter=10000` allows for enough iterations for the optimisation to converge. The `.fit(Xtrain, y)` method trains the model on the input values stored by `Xtrain` and their corresponding labels y, producing the learnt parameters.

From this, the respective model parameters were obtained:

C	Intercept (θ_0)	Feature 1 Coefficient (θ_1)	Feature 2 Coefficient (θ_2)
0.001	-0.37470508	0.00026422	-0.31740942
0.01	-0.56183597	0.01255081	-0.77406943
0.1	-0.69999814	0.03361944	-1.1347977
1	-0.73577541	0.03844468	-1.21820784
10	-0.74012781	0.03896795	-1.22807594
100	-0.74057019	0.03902203	-1.22907734
1000	-0.74061446	0.03902744	-1.22917756

Table 1. Model Parameters (θ) per Penalty Parameter (C)

For small C values ranging from 0.001 to 0.1, the coefficients are small in magnitude, reflecting a softer decision boundary that tolerates more misclassifications. As C increases (from C≥1), the parameters converge, producing a harder margin. Across all values, the magnitude of Feature 2 has a much larger coefficient magnitude than Feature 1, indicating that it is the dominant factor in the classification. This is consistent with the logistic regression model, although the scaling differs: logistic regression produced larger absolute coefficients, whereas the SVM parameters stabilise at smaller values due to its margin-maximising objective.

(ii) Just as in the logistic regression model, the linear equation derived from, $P(y = 1|x) = \frac{1}{1+e^{-\theta^T x}}$, that being, $\theta^T x = 0$, can be solved to get the decision boundary for each SVM:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$$

$$\theta_2 x_2 = -\theta_0 - \theta_1 x_1$$

$$\Rightarrow x_2 = -\frac{\theta_0 + \theta_1 x_1}{\theta_2}$$

Using this equation, the decision boundary could now be implemented through the following code:

```
svm_y_pred = svm_model.predict(Xtrain)
svm_theta0 = svm_model.intercept_[0]
svm_theta1, svm_theta2 = svm_model.coef_[0]
        #param vals for SVM Decision boundary
x1_vals = np.linspace(Xtrain.iloc[:,0].min(), Xtrain.iloc[:,0].max(), 100)
x2_vals = -(svm_theta0 + svm_theta1 * x1_vals) / svm_theta2
```

Here, `svm_` was prefixed to most variable names to differentiate them from the logistic regression variables. By deriving the classification decision boundary for each C value, the following subplots could be constructed.

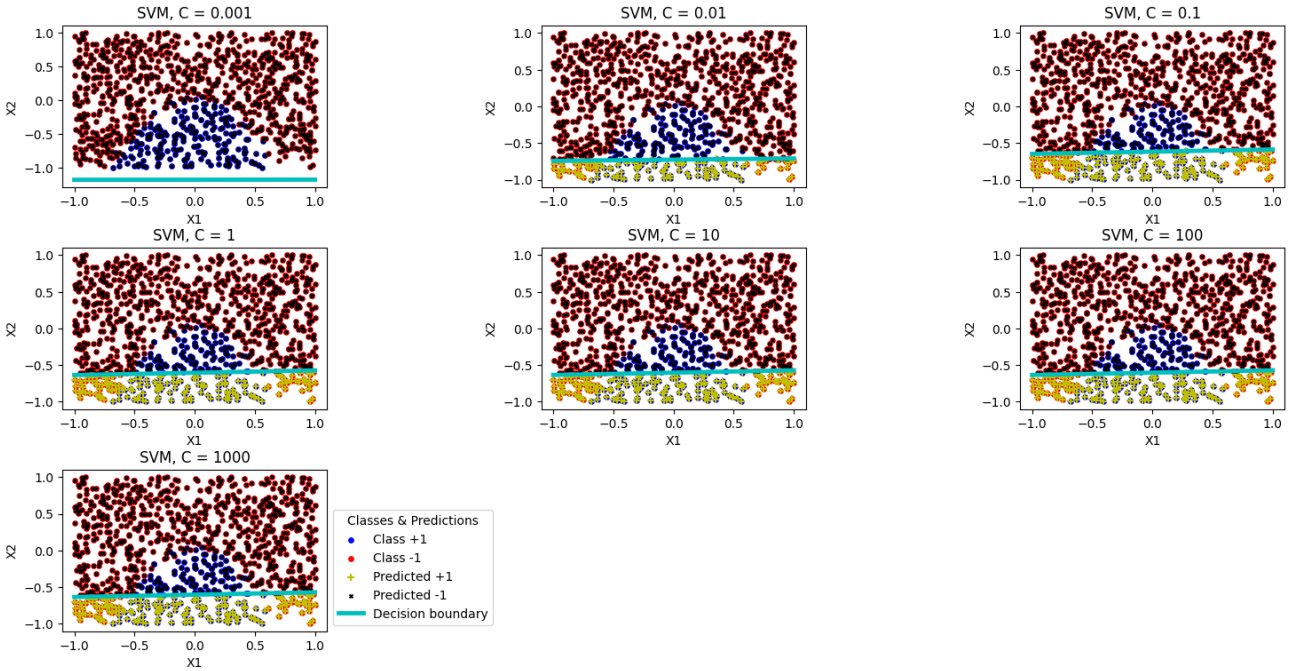

Figure 3. SVM Subplots & Decision Boundaries

(iii) The penalty parameter C controls the balance between maximising the margin of separation between data points from each class and the decision boundary and correctly classifying the training data. This optimisation can be expressed via the SVM cost function:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} max(0, 1 - y^i\theta^T x^i) + \frac{\theta^T\theta}{C}$$

Here, the first term is the hinge loss that penalises misclassified points, and the second term added to the former is the L_2 penalty on the parameters. As demonstrated in figure 3 above, while the value of C is low (C=0.001), the penalty ($\frac{\theta^T\theta}{C}$) increases, giving it more importance in the function, resulting in the parameters (θ) being closer to zero, producing a decision boundary with a weaker fit that allows more misclassifications. As C increases, the penalty is reduced, so the parameters grow in magnitude and the decision boundary adjusts to classify more of the training data correctly. Beyond a certain threshold (in this case C≥10), the parameters and predictions stabilise, showing that the model has converged to a maximum-margin separator.

(iv) The training accuracy for both models was computed using the model.score function:

```
log_acc = model.score(Xtrain, y)
print("Logistic Regression Training Accuracy:", log_acc)
for i, C in enumerate(C_values):
    svm_model = LinearSVC(C=C, max_iter=10000).fit(Xtrain, y)
    svm_acc = svm_model.score(Xtrain, y)
    print(f"SVM Training Accuracy (C={C}): {svm_acc}")
```

Which outputted the following accuracy values in the terminal:

```
Logistic Regression Training Accuracy: 0.8158158158158159
SVM Training Accuracy (C=0.001): 0.7837837837837838
SVM Training Accuracy (C=0.01): 0.8228228228228228
SVM Training Accuracy (C=0.1): 0.8148148148148148
SVM Training Accuracy (C=1): 0.8118118118118118
SVM Training Accuracy (C=10): 0.8118118118118118
SVM Training Accuracy (C=100): 0.8118118118118118
SVM Training Accuracy (C=1000): 0.8118118118118118
```

Logistic regression achieved an accuracy of 81.6% with parameters $\theta_0 = -2.14$, $\theta_1 = 0.123,$ and $\theta_2 = -3.48$. In contrast, the SVM models produced parameters that varied with C but stabilised beyond $C \geq 10$ at the approximate valves, $\theta_0 \approx -0.74,$ $\theta_1 \approx 0.039,$ and $\theta_2 \approx -1.229$. These coefficients are smaller in magnitude than those from logistic regression, reflecting the stronger margin-based regularisation imposed by SVM.

In terms of SVM accuracy, it achieved 78.4% at C=0.001, improved to a peak of 82.3% at C=0.01, and then stabilised around 81.2% for larger C values ($C \geq 1$). This stabilised SVM accuracy is comparable to logistic regression, which also sat at approximately 81.6%.

Both logistic regression and linear SVM yielded linear decision boundaries. However, the underlying data distribution has a curved overlap between the two classes, which neither model can fully capture. To capture this curved separation in the data, a nonlinear classifier such as logistic regression with polynomial features would be required.

Part C: Quadratic Logistic Regression

(i) To capture potential nonlinear relationships, the $Xtrain$ feature set is extended by adding squared terms for each original feature creating an $Xpoly$ feature set:

$$Xpoly = [X1, X2, X_1^2, X_2^2]$$

This infers that the model parameters and decision boundary are now:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 = 0$$

A quadratic logistic regression classifier could then be implemented and trained on this four-dimensional input through the following code:

```
X1_sq = X1**2
X2_sq = X2**2
X_poly = np.column_stack((X1, X2, X1_sq, X2_sq))
model_poly = LogisticRegression().fit(X_poly, y)
y_poly_pred = model_poly.predict(X_poly)
theta0_poly = model_poly.intercept_[0]
theta_poly = model_poly.coef_[0]
print("Logistic Regression with Quadratic Features")
print("Intercept (θ0):", theta0_poly)
print("Coefficients (θ1, θ2, θ3, θ4):", theta_poly)
```

The parameter values obtained from training the model are, $\theta_0 = -0.58$, $\theta_1 = 0.123$, $\theta_2 = -5.29$, $\theta_3 = -8.31$, and $\theta_4 = -0.11$. Thus, the model parameters can be denoted as:

$$\theta^T x = -0.58 + 0.123x_1 - 5.29x_2 - 8.31x_1^2 - 0.11x_2^2$$

(ii) The prediction and actual target values were plotted using the following code:

```
plt.figure(figsize=(7,7))
plt.scatter(X1[y == 1], X2[y == 1], s=40, marker='o', color='b', label='Class +1')
plt.scatter(X1[y == -1], X2[y == -1], s=40, marker='o', color='r', label='Class -1')
plt.scatter(X1[y_poly_pred == 1], X2[y_poly_pred == 1], marker='+', color='y', label='Predicted +1')
plt.scatter(X1[y_poly_pred == -1], X2[y_poly_pred == -1], s=12.5, marker='x', color='k', label='Predicted -1')
```

The quadratic logistic regression classifier was applied to the training data. The predictions were plotted against the actual labels in the original feature space.

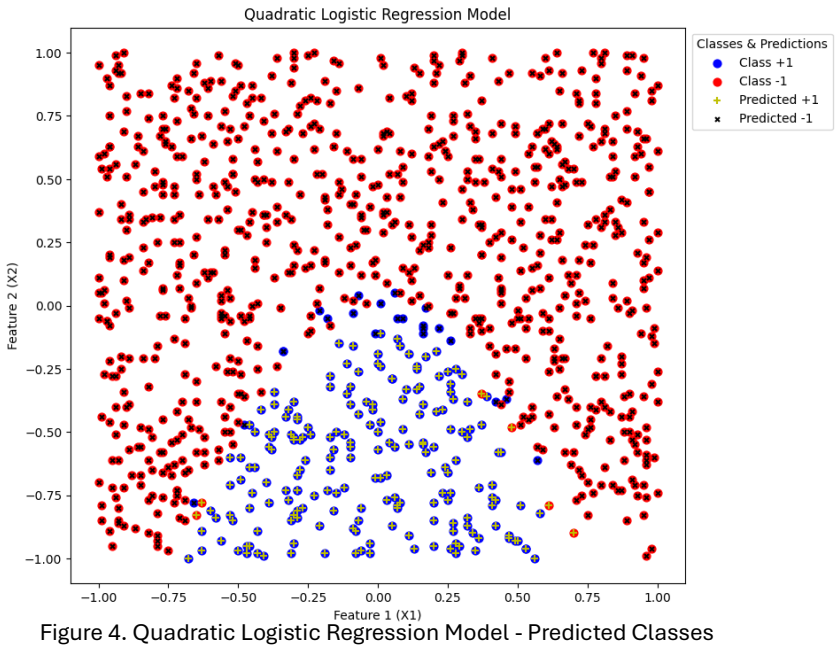

Figure 4. Quadratic Logistic Regression Model - Predicted Classes

It can be observed from figure 4 that unlike the linear models in parts (a) and (b), this quadratic model properly classifies almost all actual class labels. The training accuracy improved to 97.2% compared with the 81% accuracy reached by both linear logistic regression and the SVM. While there are still misclassified points, particularly around where the classes begin to overlap, the overall number of misclassified points was significantly reduced. This demonstrates that adding quadratic features allows the model to capture non-linear arrangements in the data, though some misclassification in the overlapping region between classes remains unavoidable.

(iii) A baseline predictor that always predicts the most common class in the dataset was implemented to serve as a reference point for assessing whether the trained model offers any improvement over a naive approach. Since the baseline is not a trained model, it does not include a built-in method for computing accuracy, so the `accuracy_score` function from `sklearn.metrics` was used:

```
from sklearn.metrics import accuracy_score
most_common_class = y.mode()[0]
baseline_pred = np.full_like(y,
fill_value=most_common_class)
baseline_acc = accuracy_score(baseline_pred, y)
```

The resulting accuracy values were as follows:

```
Quadratic Logistic Regression Training Accuracy: 0.9719719719719719
Baseline Predictor Accuracy: 0.7837837837837838
```

The trained quadratic logistic regression classifier achieved a training accuracy of 97.2%, while the baseline predictor achieved 78.4%. This demonstrates that the logistic regression classifier performs substantially better than the baseline. The baseline accuracy reflects the underlying class imbalance, as it ignores all feature information. In contrast, the quadratic logistic regression model effectively captures the nonlinear relationship between features and the target variable, yielding a much higher accuracy and a curved decision boundary that closely aligns with the actual class separation.

(iv) A quadratic equation can be derived in terms of x_2 from the quadratic logistic regression classifier decision boundary:

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 = 0$$

Rearranging gives a standard quadratic form in x_2:

$$(\theta_4)x_2^2 + (\theta_2)x_2 + (\theta_0 + \theta_1 x_1 + \theta_3 x_1^2) = 0$$

Where:

$$a = \theta_4, \qquad b = \theta_2, \qquad c = \theta_0 + \theta_1 x_1 + \theta_3 x_1^2.$$

Using the quadratic formula, the boundary can be expressed as:

$$x_2 = \frac{-\theta_2 \pm \sqrt{\theta_2^2 - 4(\theta_4)(\theta_0 + \theta_1 x_1 + \theta_3 x_1^2)}}{2(\theta_4)}$$

To create a decision boundary using the above quadratic formula the following code was implemented:

```
disc = theta_poly[1]**2 - 4*theta_poly[3]*(theta0_poly + theta_poly[0]*x1_vals + theta_poly[2]*x1_vals**2)
valid = disc >= 0
x1_valid = x1_vals[valid]
disc_valid = np.sqrt(disc[valid])
#x2_pos = (-theta_poly[1] + disc_valid) / (2*theta_poly[3])
x2_neg = (-theta_poly[1] - disc_valid) / (2*theta_poly[3])
#plt.plot(x1_valid, x2_pos, 'g--', label='Decision boundary (+ branch)')
plt.plot(x1_valid, x2_neg, 'g--', label='Decision boundary (- branch)')
```

The discriminant ($b^2 - 4ac$), denoted as `disc`, is calculated for each x_1 value in `x1_vals`. To make sure the curve remains `valid` for plotting, only real-valued points (where the discriminant ≥ 0) are retained. The corresponding x_2 values are then computed using both the positive and negative roots of the quadratic equation, producing two sets of values: `x2_pos` and `x2_neg`. Plotting each against x_1 constructs two possible curves representing the decision boundary. Among these, the negative branch aligned best with the class separation, forming the true quadratic decision boundary.

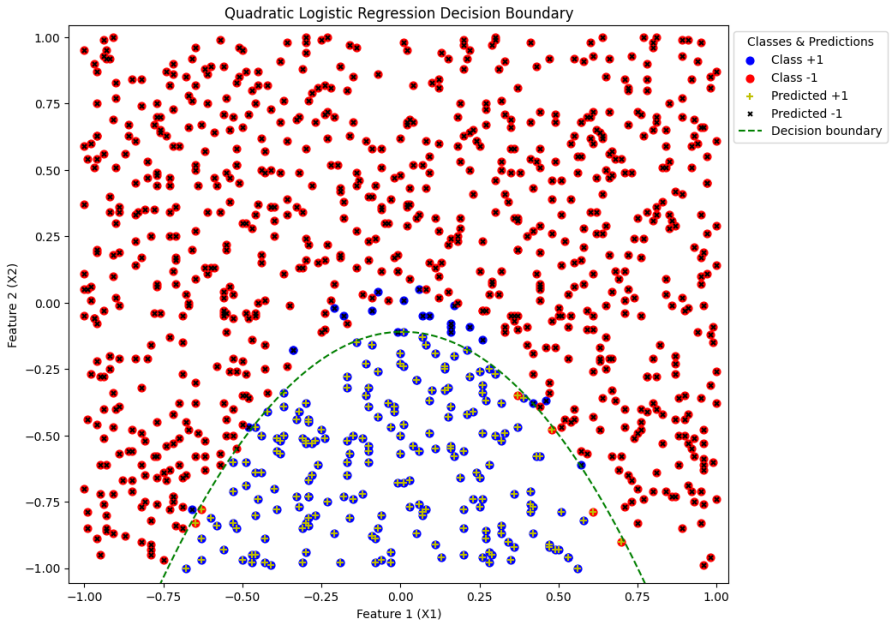

Figure 5. Quadratic Logistic Regression Model - Predicted Classes & Decision Boundary

As seen in figure 5, the quadratic logistic regression model produced a curved decision boundary, which better follows the overlapping region between the two classes while separating them compared to the linear decision boundaries produced in parts (a) and (b). However, as mentioned in c(ii), some misclassification in the overlapping region between classes remains unavoidable, apparent by the misclassified class +1 datapoints above the decision boundary and misclassified class -1 datapoints below the decision boundary.

Appendix

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

file = pd.read_csv('week2.csv', header=None)

X1 = file.iloc[:,0] #select all rows in col1

X2 = file.iloc[:,1] #col2

y = file.iloc[:, 2] #col3

'''-------------------------a(i-iii)-----------------------'''

#logistic regression model

from sklearn.linear_model import LogisticRegression

Xtrain = file.iloc[:,0:2] #col1&2

model = LogisticRegression().fit(Xtrain, y)

y_pred = model.predict(Xtrain)

print("Intercept (θ0):", model.intercept_)

print("Coeffs (θ1, θ2):", model.coef_)

#print("(θ1, θ2): (", model.coef_[0,0], ", ", model.coef_[0,1], ")")

theta0 = model.intercept_[0]

theta1, theta2 = model.coef_[0]


#param vals for decision boundary

x1_vals = np.linspace(Xtrain.iloc[:,0].min(), Xtrain.iloc[:,0].max(), 100)

x2_vals = -(theta0 + theta1 * x1_vals) / theta2


#plot

plt.figure(figsize=(7,7))

    #training data

plt.scatter(X1[y == 1], X2[y == 1], s=40, marker='o', color='b', label='Class +1')

plt.scatter(X1[y == -1], X2[y == -1], s=40, marker='o', color='r', label='Class -1')

    #predictions

plt.scatter(Xtrain[y_pred==1].iloc[:,0], Xtrain[y_pred==1].iloc[:,1],  marker='+', color='y', label='Predicted
+1')

plt.scatter(Xtrain[y_pred==-1].iloc[:,0], Xtrain[y_pred==-1].iloc[:,1], s=12.5, marker='x', color='k',
label='Predicted -1')

    #dec boundary

plt.plot(x1_vals, x2_vals, 'k--', label='Decision boundary')

    #lables & legend

plt.xlabel('Feature 1 (X1)')

plt.ylabel('Feature 2 (X2)')

plt.title('2D Feature Value Scatter Plot')

plt.legend(title="Target Class", loc = 'upper right', bbox_to_anchor=(1.20, 1))
```

```
plt.tight_layout()
plt.show()

'''-------------------------b(i-iii)-----------------------'''
#SVC
from sklearn.svm import LinearSVC
C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

#subplot grid (7 plots, 3x3 grid)
fig, axes = plt.subplots(3, 3, figsize=(9, 9))
axes = axes.ravel()  #flatten to index with [i]

    #logistic regression accuracy b(iv)
log_acc = model.score(Xtrain, y)
print("Logistic Regression Training Accuracy:", log_acc)

#SVM plot per C loop
for i, C in enumerate(C_values):
    svm_model = LinearSVC(C=C, max_iter=10000).fit(Xtrain, y)
    svm_y_pred = svm_model.predict(Xtrain)

        #SVM accuracies across C values (b(iv))
    svm_acc = svm_model.score(Xtrain, y)
    print(f"SVM Training Accuracy (C={C}): {svm_acc}")

    #print(f"\nC = {C}")
    #print("Intercept (00):", svm_model.intercept_)
    #print("Coeffs (01, 02):", svm_model.coef_)
    svm_theta0 = svm_model.intercept_[0]
    svm_theta1, svm_theta2 = svm_model.coef_[0]
        #param vals for SVM Decision boundary
    x1_vals = np.linspace(Xtrain.iloc[:,0].min(), Xtrain.iloc[:,0].max(), 100)
    x2_vals = -(svm_theta0 + svm_theta1 * x1_vals) / svm_theta2

    #subplots
    ax = axes[i]
        #training data
    ax.scatter(X1[y == 1], X2[y == 1], s=15, marker='o', color='b', label='Class +1')
    ax.scatter(X1[y == -1], X2[y == -1], s=15, marker='o', color='r', label='Class -1')
```

```
        #predictions
    ax.scatter(Xtrain[svm_y_pred == 1].iloc[:,0], Xtrain[svm_y_pred == 1].iloc[:,1],
               marker='+', color='y', label='Predicted +1')
    ax.scatter(Xtrain[svm_y_pred == -1].iloc[:,0], Xtrain[svm_y_pred == -1].iloc[:,1],
               s=7.5, marker='x', color='k', label='Predicted -1')
        #dec boundary
    ax.plot(x1_vals, x2_vals, 'c-', linewidth = 3.5, label='Decision boundary')
        #lables
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')
    ax.set_title(f'SVM, C = {C}')
        #legend added to the last subplot only
    if i == len(C_values) - 1:
        ax.legend(title="Classes & Predictions", loc='lower right',  bbox_to_anchor=(1.65, 0))

#hide unused subplots (since only use 7/9)
for j in range(len(C_values), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

'''-------------------------c(i-iv)-----------------------'''
#poly features
X1_sq = X1**2
X2_sq = X2**2

#combine feats & poly feats into 4 column matrix
X_poly = np.column_stack((X1, X2, X1_sq, X2_sq))

#quadratic logistic regression model
model_poly = LogisticRegression().fit(X_poly, y)
y_poly_pred = model_poly.predict(X_poly)

#parameters
theta0_poly = model_poly.intercept_[0]
theta_poly = model_poly.coef_[0]

print("Logistic Regression with Quadratic Features")
print("Intercept (00):", theta0_poly)
```

```
print("Coefficients (01, 02, 03, 04):", theta_poly)


#plot
plt.figure(figsize=(7,7))
    #trainning data
plt.scatter(X1[y == 1], X2[y == 1], s=40, marker='o', color='b', label='Class +1')
plt.scatter(X1[y == -1], X2[y == -1], s=40, marker='o', color='r', label='Class -1')
    #predictions
plt.scatter(X1[y_poly_pred == 1], X2[y_poly_pred == 1], marker='+', color='y', label='Predicted +1')
plt.scatter(X1[y_poly_pred == -1], X2[y_poly_pred == -1], s=12.5, marker='x', color='k', label='Predicted -1')

#quadratic logistic regression accuracy
quad_acc = model_poly.score(X_poly, y)
print("Quadratic Logistic Regression Training Accuracy:", quad_acc)
#baseline classifier (predicts most common class in dataset)
most_common_class = y.mode()[0]
baseline_pred = np.full_like(y, fill_value=most_common_class)
    #since there's trained model for bsln pred
from sklearn.metrics import accuracy_score
baseline_acc = accuracy_score(baseline_pred, y)
print("Baseline Predictor Accuracy:", baseline_acc)

#quad logistic regression dec boundary(quadratic EQ)
#discriminant (b^2-4ac)
disc = theta_poly[1]**2 - 4*theta_poly[3]*(theta0_poly + theta_poly[0]*x1_vals + theta_poly[2]*x1_vals**2)
#keep only valid discriminants(disc >= 0)
valid = disc >= 0
x1_valid = x1_vals[valid]
disc_valid = np.sqrt(disc[valid])

#(+/-) x2 curves
#x2_pos = (-theta_poly[1] + disc_valid) / (2*theta_poly[3])   #doesn't fit data
x2_neg = (-theta_poly[1] - disc_valid) / (2*theta_poly[3])

#plot
#plt.plot(x1_valid, x2_pos, 'g--', label='Decision boundary')   #doesn't fit data
plt.plot(x1_valid, x2_neg, 'g--', label='Decision boundary')
    #lables & legend
plt.xlabel('Feature 1 (X1)')
plt.ylabel('Feature 2 (X2)')
```

```
plt.title('Quadratic Logistic Regression Decision Boundary')
plt.legend(title="Classes & Predictions", loc='upper right', bbox_to_anchor=(1.25, 1))


plt.tight_layout()
plt.show()
```